Procedimientos Almacenados

Funcionalidad y Aplicación



Bloques de código SQL reutilizables que se guardan en el servidor y pueden ejecutarse cuando se necesite se necesite

```
CREATE PROCEDURE nombre_procedimiento (parámetros)
```

19 de octubre de 2025

¿Qué es un Procedimiento Almacenado?

Un procedimiento almacenado es un bloque de código SQL que se guarda directamente en el servidor de la base de datos.

Funciona como una "función" que encapsula lógica reutilizable, permitiendo su ejecución bajo demanda y facilitando la organización y eficiencia en la gestión de datos.



Características Principales:

Reutilizable

Se puede llamar y ejecutar varias veces desde diferentes aplicaciones y usuarios

Almacenamiento en Servidor

El código reside directamente en la base de datos, lo que facilita su gestión y seguridad

Seguridad

Permite controlar el acceso a los datos mediante permisos específicos y centraliza la lógica de negocio

Ventajas y Utilidades

Los procedimientos almacenados ofrecen múltiples beneficios que optimizan la gestión y el rendimiento de las bases de datos:



Automatización de Tareas Repetitivas

Permiten programar y ejecutar secuencias de operaciones de forma recurrente, como la limpieza de registros antiguos o la generación de informes periódicos.



Aplicación de Reglas de Negocio

Centralizan la lógica de negocio directamente en la base de datos, asegurando que las reglas se apliquen de manera consistente en todas las interacciones con los datos.



Centralización de Lógica

Mueven la lógica que antes residía en la aplicación al servidor de la base de datos, facilitando el mantenimiento y la actualización del código.



Mejora del Rendimiento

Al ejecutar el código directamente en el servidor, se reduce el tráfico de red entre el cliente y el servidor, lo que se traduce en una mayor eficiencia y velocidad.

💡Ejemplo: Un procedimiento almacenado puede validar datos de entrada antes de insertarlos en una tabla, aplicando las reglas de negocio de manera consistente.

Gestión de Permisos

Permisos para Crear

Para crear un procedimiento almacenado, el usuario debe tener el permiso CREATE PROCEDURE:

```
GRANT CREATE PROCEDURE TO 'usuario'@'host';
```

Este permiso se otorga una vez y se aplica a todos los procedimientos que el usuario pueda crear

Permisos para Ejecutar

Para ejecutar un procedimiento almacenado, se requiere el permiso EXECUTE sobre el procedimiento específico:

```
GRANT EXECUTE ON PROCEDURE esquema.nombre TO 'usuario'@'host';
```

```
GRANT EXECUTE ON PROCEDURE base_de_datos.nombre_procedimiento TO 'usuario'@'host';
```

Este permiso debe otorgarse para cada procedimiento específico que se desee ejecutar

Permisos de Acceso a Tablas

Si el procedimiento interactúa con tablas (realizando SELECT, INSERT, UPDATE, DELETE), el usuario que lo ejecuta también debe tener los permisos correspondientes sobre esas tablas.

SELECT INSERT UPDATE DELETE

Los permisos de tabla se verifican en tiempo de ejecución, no cuando se crea el procedimiento

La gestión adecuada de permisos es crucial para la seguridad de la base de datos

Sintaxis de Creación

Consola / phpMyAdmin / Workbench

En el cliente

Es necesario utilizar **DELIMITER**

```
DELIMITER //CREATE PROCEDURE
nombre_procedimiento (parámetros)BEGIN--
sentencias SQLEND //DELIMITER ;
```

El punto y coma (;) se interpreta como el final de una sentencia SQL

MariaDB 10.4

En el servidor

No soporta la cláusula **CREATE OR REPLACE PROCEDURE**.

```
DROP PROCEDURE IF EXISTS
nombre_procedimiento;CREATE PROCEDURE
nombre_procedimiento (...)BEGIN-- nuevo
cuerpoEND;
```

Solución recomendada: eliminar y recrear el procedimiento

```
DELIMITER //

-- 1. Si existe la versión vieja, la elimina
DROP PROCEDURE IF EXISTS consultaincidencia //

-- 2. Crea la versión limpia
CREATE PROCEDURE consultaincidencia(IN paramId INT)
BEGIN
    DECLARE Des VARCHAR(30);
    SELECT descripcion INTO Des FROM incidencias WHERE id = paramId;
    SELECT Des;
END //

-- 3. Devuelve el delimitador a su estado normal
DELIMITER ;
```

Importante: La sintaxis puede variar según el entorno de desarrollo. Asegúrate de usar la sintaxis adecuada para tu entorno.

Ejecución y Eliminación

Ejecución

La ejecución de un procedimiento almacenado se realiza mediante la instrucción CALL, seguida del nombre del procedimiento y los valores de sus parámetros.

Consola mysql

```
-- Para ver el valor de un parámetro OUTCALL
nombre_procedimiento(valor1, valor2, @resultado);
SELECT @resultado;
```

Workbench

```
CALL nombre_procedimiento(valor1, valor2, @resultado);
SELECT @resultado;
```

En Workbench, la visualización de los resultados de los parámetros OUT es automática (si usamos los botones y asistentes que nos pueden mostrar los resultados de salida de forma visual sin escribir código, si no, tendríamos que poner en el editor de texto la sentencia `SELECT @resultado;`), mientras que en la consola es necesario usar la sentencia SELECT adicional.

Eliminación

Para eliminar un procedimiento almacenado que ya no es necesario, se utiliza la instrucción DROP PROCEDURE. Es una buena práctica incluir IF EXISTS para evitar errores.

```
DROP PROCEDURE nombre_procedimiento;
</>

DROP PROCEDURE IF EXISTS nombre_procedimiento;
```

Importante

Al eliminar un procedimiento, se pierde toda la lógica encapsulada en él. Asegúrate de que no esté siendo utilizado por ninguna aplicación antes de eliminarlo.

Errores Comunes y Soluciones

La gestión de errores es crucial en el desarrollo de procedimientos almacenados. Identificar la causa y aplicar la solución adecuada puede ahorrar tiempo y esfuerzo.

ERROR 1304

Ya existe el procedimiento

Solución:

Usar `DROP PROCEDURE IF EXISTS` antes de crearlo

ERROR 1044

Permiso denegado

Solución:

Conceder permiso `CREATE PROCEDURE` al usuario

ERROR 1064

Error de sintaxis

Solución:

Revisar delimitadores y estructura del bloque

ERROR 1370

No se puede ejecutar

Solución:

Falta permiso `EXECUTE` sobre el procedimiento

Consejo: Siempre verifica los permisos y la sintaxis antes de crear o ejecutar procedimientos almacenados.

TIPOS DE VARIABLES

Tipo de Variable	Descripción	Ejemplo
INT / INTEGER	Números enteros	DECLARE contador INT;
DECIMAL / FLOAT	Números con decimales	DECLARE precio DECIMAL(10,2);
CHAR(n) / VARCHAR(n)	Texto de longitud fija o variable	DECLARE nombre VARCHAR(50);
TEXT / TINYTEXT / MEDIUMTEXT / LONGTEXT	Texto largo	DECLARE descripcion TEXT;
DATE	Fecha (año-mes-día)	DECLARE fecha_nacimiento DATE;
TIME	Hora (hh:mm:ss)	DECLARE hora_inicio TIME;
DATETIME / TIMESTAMP	Fecha y hora combinadas	DECLARE creado_en DATETIME;
BOOLEAN / BOOL	Valor lógico (TRUE/FALSE)	DECLARE activo BOOLEAN;
BLOB / TINYBLOB / MEDIUMBLOB / LONGBLOB	Datos binarios (archivos, imágenes, etc.)	DECLARE archivo BLOB;
ENUM	Lista de valores predefinidos. (Elección ÚNICA y excluyente)	DECLARE estado ENUM('activo','inactivo');
SET	Conjunto de valores predefinidos.(Elección MÚLTIPLE y combinable)	DECLARE permisos SET('leer','escribir','borrar');

CÓMO INTEGRAMOS LAS CONSULTAS.

Ejemplo pasando parámetro

```
DELIMITER //
CREATE PROCEDURE consultaincidencia(IN paramId INT)
BEGIN
 Declare Des varchar(30);
 select descripcion  into Des  from incidencias  where id = paramId;
 select Des;
END //
```

Ejemplo sin pasa parámetro

```
DELIMITER //
      CREATE PROCEDURE Tablaincidencia()
          BEGIN select * from incidencias;
      END //
DELIMITER ;
```

Ejercicios. Vais a usar la base de datos que habéis creado en el ejercicio del Reto. datashield_db. Os conectais con el usuario root, para poder realizar los siguientes ejercicios. Para hacer un seguimiento del trabajo. Vamos hacer lo siguiente:

1.-> Subir, en único fichero SQL los procedimientos que habéis realizado. Debéis indicar el número de ejercicio que corresponde.

2.-> Corregiremos, vuestros ejercicios, en clase, proyectando en la pizarra digital y explicando qué habéis hecho.

Ejercicio 1: Mostrar incidencias con nombre del sistema

Enunciado: Crear un procedimiento que muestre **todas** las incidencias(id, descripción) junto con el nombre del sistema afectado, de la tabla sistema.

Ejercicio 2: Contar incidencias por nivel de gravedad

Enunciado: Crear un procedimiento que reciba un nivel de gravedad y devuelva cuántas incidencias hay con ese nivel. Siendo los niveles de gravedad que pueda tomar lo siguientes: baja, media, alta. Debes usar COUNT(*) INTO total

Ejercicio 3: Cambiar estado de una incidencia

Enunciado: Crear un procedimiento que reciba el ID de una incidencia y un nuevo estado, y actualice el estado. Siendo los estados posible: en curso, resuelta.

Ejercicio 4: Mostrar incidencias recientes

Enunciado: Crear un procedimiento que muestre las incidencias registradas en los últimos 7 días. Para calcular la diferencia de días, se tiene que utilizar *WHERE fecha >= NOW() - INTERVAL 7 DAY;*

Ejercicio 5: Insertar nueva incidencia

Enunciado: Crear un procedimiento que reciba los datos de una incidencia (sistema_id, descrip, gravedad, estado, fecha) y la inserte en la tabla.